

HomeKey

Smart-Schlüsselbrett mit Wiener-Linien-Anzeige, Reed-Sensorik und Home-Assistant-Integration

Projektdokumentation

Autor: Marc

Stand: 18. Juni 2026

Inhaltsverzeichnis

1. Projektübersicht

HomeKey ist ein selbst entwickeltes “smartes” Schlüsselbrett für eine Wohngemeinschaft (Kilian, Marc, Patrik). Es kombiniert drei Funktionsbereiche in einem System:

- Eine Live-Abfahrtsanzeige der Wiener Linien (U6) für zwei Haltestellen (Floridsdorf, Siebenhirten) auf einem Lilygo T-Display S3.
- Drei Reed-Kontakt-Sensoren, die erkennen, ob der Schlüssel von Kilian, Marc bzw. Patrik am Brett hängt oder mitgenommen wurde.
- Eine Home-Assistent/MQTT-Infrastruktur (in Docker betrieben), die die Sensordaten empfängt, auswertet und in einem Dashboard visualisiert, inklusive einer Fern-Ein/Aus-Schaltung für das Display.

Das Projekt ist iterativ über mehrere Entwicklungs-Sessions gewachsen. Kapitel 3 dokumentiert diese Entwicklung chronologisch, da sich Pinbelegung, Netzwerk-Konfiguration und Funktionsumfang (insbesondere die LED-Leisten) im Projektverlauf mehrfach geändert haben.

2. Hardware

2.1 Komponentenliste

Komponente	Funktion
Lilygo T-Display S3 (ESP32-S3)	Zentrale Steuereinheit: WLAN, MQTT-Client, ST7789-TFT-Ansteuerung (170×320, 8-Bit Parallel)
3× Reed-Kontakt-Sensor	Erkennung, ob Schlüssel von Kilian / Marc / Patrik vorhanden ist
KY-021-Modul (Patrik)	Abweichendes Sensormodul mit eigener Pull-down-Logik – benötigt invertierte Auswertung
WS2812B-LED-Streifen (2×)	Geplant: 6 LEDs für Schlüssel-Anwesenheit (GPIO 2), 35 LEDs für Abfahrtszeiten-Farbcodierung (GPIO 16) – Status siehe Kap. 7
Externes 5-V-Netzteil	Stromversorgung der LED-Streifen, GND gemeinsam mit ESP32
Docker-Host (PC)	Betreibt Home Assistant, Mosquitto-Broker, ESPHome, Nginx

Tabelle 1: Verwendete Hardware-Komponenten

2.2 Display-Pinbelegung (TFT_eSPI User_Setup.h)

Das Lilygo T-Display S3 nutzt ein 8-Bit-Parallel-Interface (kein SPI). Treiber: ST7789, Auflösung 170×320.

Signal	GPIO
CS	6
DC	7

RST	5
WR	8
RD	9
D0-D7 (Datenbus)	39, 40, 41, 42, 45, 46, 47, 48
Backlight (BL)	38 – wird zusätzlich per MQTT-Schalter gesteuert (siehe Kap. 5)

Tabelle 2: TFT-Pinbelegung laut User_Setup.h

2.3 Reed-Sensoren

Person	GPIO	Invertiert?	Bemerkung
Kilian	10	Nein	Standard-Reedkontakt mit INPUT_PULLUP, LOW = Schlüssel vorhanden
Marc	12	Nein	Standard-Reedkontakt mit INPUT_PULLUP
Patrik	11	Nein (aktueller Stand)	KY-021-Modul – siehe Hinweis unten

Tabelle 3: Aktuelle Pin- und Logikzuordnung (Main_code.ino)

Hinweis zum KY-021 (Patrik): Im Gegensatz zu den beiden anderen, einfachen Reed-Kontakten besitzt das KY-021-Modul einen eingebauten Pull-down-Widerstand. Dadurch musste die Auswertungslogik für diesen Pin zeitweise invertiert werden (`invertedPins[2] = true`, `pinMode INPUT` statt `INPUT_PULLUP`). Im Projektverlauf wurde dieser Wert mehrfach umgestellt – siehe Entwicklungs-Historie in Kapitel 3.4. Im aktuellen Code steht er auf `false`.

Sicherheitsproblem (3,3 V vs. 5 V): Patrik's KY-021-Modul war zwischenzeitlich versehentlich an 5 V statt 3,3 V angeschlossen – ein Risiko für die GPIO-Eingänge des ESP32 (nicht 5 V-tolerant). Bis zum Nachlöten auf 3,3 V wurde der Pin softwareseitig vollständig deaktiviert (`disabledPins[2] = true`), sodass weder `pinMode()` noch `digitalRead()` für diesen Pin ausgeführt wurden und stattdessen dauerhaft "OFF" publiziert wurde.

2.4 LED-Streifen (in Entwicklung)

Zusätzlich zur reinen Sensor-/Anzeige-Funktion war eine optische Rückmeldung per WS2812B-LED-Streifen geplant:

- 6 LEDs auf GPIO 2: Schlüssel-Anwesenheit, 2 LEDs pro Person, leuchten weiß wenn der Schlüssel am Brett hängt.
- 35 LEDs auf GPIO 16: Farbcodierte Abfahrtszeiten-Anzeige, aufgeteilt in 17 LEDs für Floridsdorf und 18 LEDs für Siebenhirten (Rot/Gelb/Grün je nach Countdown).
- Versorgung über ein externes 5-V-Netzteil; GND von Netzteil, beiden LED-Streifen und ESP32 müssen gemeinsam verbunden sein (gemeinsame Masse).

Verkabelungs-Checkliste (Lötarbeiten):

1. PSU + → LED VCC (beide Streifen)

2. PSU – → LED GND (beide Streifen)
3. PSU – → ESP32 GND (gemeinsame Masse)
4. ESP32 GPIO 2 → Data-Leitung Streifen 1 (6 LEDs)
5. ESP32 GPIO 16 → Data-Leitung Streifen 2 (35 LEDs)
6. Reparatur eines abgebrochenen Lötkontakts am LED-Streifen

Aktueller Status: Die LED-Erweiterung (FastLED-Code für 6+35 LEDs) wurde in einer Session entwickelt und verdrahtet, ist aber noch nicht in den aktuell produktiven Code (Main_code.ino) übernommen. In einem früheren Entwicklungsstrang wurden die LEDs sogar komplett aus dem Code entfernt, nachdem ein Defekt auftrat (“alle LEDs sind leider kaputt”), und durch ein Home-Assistant-Dashboard als visuelles Feedback ersetzt (siehe Kapitel 5 und 7).

3. Softwarearchitektur

3.1 Entwicklungs-Historie (chronologisch)

Das Projekt wurde nicht in einem Zug entwickelt, sondern durchlief mehrere Phasen mit unterschiedlichen Code-Versionen, WLAN-Netzwerken und Hardware-Konfigurationen:

Phase	Beschreibung
1. Reine Abfahrtsanzeige	Einzelnes ESP32-S3-Sketch zeigt nur Wiener-Linien-Abfahrtszeiten an (zunächst eine Haltestelle, dann zwei; Layout von vertikal auf horizontal umgestellt).
2. Docker/MQTT-Infrastruktur	Aufbau von Home Assistant, Mosquitto-Broker, ESPHome und Nginx via docker-compose.
3. Kombierter Sketch	Zusammenführung von Sensorik (3 Reed-Kontakte), MQTT-Publishing und WS2812B-LED-Streifen (6+35 LEDs) in einem Sketch. Diverse Compile-Fehler behoben (fehlendes schließendes Anführungszeichen beim WLAN-Passwort, JSON-Verschachtelungslimit).
4. LED-Ausfall & Entfernung	LED-Streifen meldeten Hardwaredefekt (“alle LEDs sind leider kaputt”); FastLED-Code vollständig entfernt; Patrik-Invertierung auf false korrigiert; stattdessen Home-Assistant-Dashboard mit Tile-Cards als visuelles Feedback aufgebaut.
5. Display-Schalter	MQTT-gesteuerter Ein/Aus-Schalter für das Display ergänzt (Topic schluesselbrett/display/set, schaltet Backlight an GPIO 38).
6. Erneuter LED-Versuch (parallele Session)	Erneuter Versuch, die LED-Streifen mit externem Netzteil wieder einzubauen (Verkabelung, gemeinsame Masse, Wiring-Diagramm) – Firmware-Erweiterung wurde entworfen, aber zum Zeitpunkt dieser Dokumentation noch nicht in Main_code.ino zusammengeführt.

Tabelle 4: Entwicklungs-Phasen des HomeKey-Projekts

3.2 Aktueller Code – Main_code.ino

Der aktuell produktive Code liegt als einzelne Datei Main_code.ino vor und vereint Display, Sensorik und MQTT (ohne LEDs, Stand Kapitel 7). Funktionsüberblick:

Funktion	Aufgabe
----------	---------

setup()	Initialisiert Serial, Backlight-Pin, Sensor-Pins, TFT-Display, WLAN- und MQTT-Verbindung
loop()	Prüft WLAN/MQTT-Verbindung, ruft checkSensors() und fetchDepartures() auf, zeichnet Display neu, wartet nicht-blockierend 30 s
fetchDepartures()	HTTPS-GET an die Wiener-Linien-OGD-Echtzeit-API, JSON-Parsing mit ArduinoJson (NestingLimit 15)
drawRow() / getCountdownColor()	Zeichnet eine Haltestellen-Zeile; färbt Countdown rot (≤ 2 min), gelb (≤ 4 min) oder grün
checkSensors() / forcePublishAllStates()	Liest die drei Reed-Pins (mit Entprellung 50 ms) und publiziert Statusänderungen per MQTT
setupMQTTDiscovery()	Sendet Home-Assistant-MQTT-Auto-Discovery-Konfiguration für 3 binary_sensor-Entitäten und 1 switch-Entität
callback()	Empfängt MQTT-Befehle vom Dashboard-Schalter und schaltet das Display-Backlight (GPIO 38) ein/aus
reconnectMQTT()	Stellt bei Verbindungsabbruch die MQTT-Verbindung wieder her und führt Discovery erneut aus

Tabelle 5: Funktionsübersicht Main_code.ino

3.3 MQTT-Topics & Home-Assistant Auto-Discovery

Topic	Richtung	Zweck
homeassistant/binary_sensor/<Name>/config	ESP32 → HA	Auto-Discovery für je einen Anwesenheitssensor (Kilian, Marc, Patrik)
schluesselbrett/<Name>/state	ESP32 → HA	ON/OFF-Status, ob Schlüssel vorhanden ist
homeassistant/switch/homekey_display/config	ESP32 → HA	Auto-Discovery für den Display-Schalter
schluesselbrett/display/state	ESP32 → HA	Aktueller Display-Zustand (ON/OFF)
schluesselbrett/display/set	HA → ESP32	Befehl zum Ein-/Ausschalten des Displays

Tabelle 6: MQTT-Topic-Übersicht

4. Infrastruktur (Docker)

4.1 docker-compose.yml

Vier Container orchestrieren die Backend-Infrastruktur:

```
services:
  homeassistant:
    container_name: homeassistant
    image: "ghcr.io/home-assistant/home-assistant:stable"
    volumes:
```

```

- "C:/Users/Startklar/OneDrive - tgm - Die Schule der Technik/Desktop/HomeKey/HomeKey/hass-config:/config"
- /etc/localtime:/etc/localtime:ro
restart: always
ports:
- "8123:8123"

mqtt:
  container_name: mosquitto
  image: eclipse-mosquitto:latest
  ports:
  - "1883:1883"
  - "9001:9001"
  volumes:
  - "C:/Users/Startklar/OneDrive - tgm - Die Schule der
Technik/Desktop/HomeKey/HomeKey/mosquitto/config:/mosquitto/config"
  - "C:/Users/Startklar/OneDrive - tgm - Die Schule der
Technik/Desktop/HomeKey/HomeKey/mosquitto/data:/mosquitto/data"
  - "C:/Users/Startklar/OneDrive - tgm - Die Schule der
Technik/Desktop/HomeKey/HomeKey/mosquitto/log:/mosquitto/log"
  restart: always

esphome:
  container_name: esphome
  image: ghcr.io/esphome/esphome
  volumes:
  - "C:/Users/Startklar/OneDrive - tgm - Die Schule der Technik/Desktop/HomeKey/HomeKey/esphome-config:/config"
  - /etc/localtime:/etc/localtime:ro
  restart: always
  ports:
  - "6052:6052"

webserver:
  image: nginx:latest
  container_name: mein_nginx_server
  ports:
  - "8080:80"
  volumes:
  - "C:/Users/Startklar/OneDrive - tgm - Die Schule der
Technik/Desktop/HomeKey/HomeKey/html:/usr/share/nginx/html"
  restart: always

```

Listing 1: docker-compose.yml

- homeassistant – Home Assistant Core, Port 8123, persistentes Config-Volume
- mqtt (mosquitto) – MQTT-Broker, Ports 1883 (MQTT) und 9001 (WebSocket)
- esphome – ESPHome-Dashboard zur Geräteverwaltung, Port 6052
- webserver (nginx) – statischer Webserver, Port 8080

4.2 Mosquitto-Konfiguration

```

listener 1883
allow_anonymous true

```

Listing 2: mosquitto/config/mosquitto.conf – Listener auf Port 1883, anonyme Verbindungen erlaubt (geeignet für ein abgeschlossenes Heimnetz)

5. Home-Assistant Dashboard

Da die LED-Rückmeldung zeitweise ausfiel, wurde ein eigenes Dashboard "HomeKey Zentrale" als Sections-Layout angelegt:

- Tile-Cards für `binary_sensor.kilian` / `marc` / `patrik` mit `mdi:key`-Icon und aktivierter “Zustand einfärben”-Funktion (grün = vorhanden, grau = abwesend).
- Ein Weather-Badge in der Kopfzeile.
- Eine Markdown-Karte mit Jinja-Templating für einen humorvollen “WG-Status”-Spruch (mit alternativen, neutraleren Formulierungen).
- Eine Webpage/Markdown-Karte mit Wiener-Linien-Informationen.
- Ein Tile- bzw. Schalter-Eintrag für den Display-Ein/Aus-Schalter (`schluesselbrett/display/set`).

6. Troubleshooting-Historie

Problem	Lösung
JSON-Fehler “TooDeep” beim Parsen der Wiener-Linien-API	<code>DeserializationOption::NestingLimit(15)</code> gesetzt
Compile-Fehler <code>tft.ORANGE</code>	Korrigiert zu <code>TFT_ORANGE</code>
Compile-Fehler durch fehlendes schließendes Anführungszeichen im WLAN-Passwort	Anführungszeichen ergänzt
Upload-Fehler “Write timeout” / USB nicht erkannt	BOOT-Taste beim Einstecken halten; COM-Port durch Aus-/Einstecken verifiziert
Patrik-Sensor zeigte invertierten Status in Home Assistant	<code>invertedPins[2]</code> zwischen <code>true/false</code> umgestellt, je nach Modul-Generation/Verkabelung
KY-021 (Patrik) versehentlich an 5 V statt 3,3 V angeschlossen	Pin softwareseitig deaktiviert (<code>disabledPins[2] = true</code>) bis zum Nachlöten auf 3,3 V
LED-Streifen meldeten Defekt (“alle LEDs sind leider kaputt”)	FastLED-Code vollständig entfernt; Home-Assistant-Dashboard als Ersatz-Feedback aufgebaut
WLAN-/MQTT-Wechsel für Schulnetzwerk (Präsentation)	SSID/Passwort und <code>mqtt_server</code> -IP im Code angepasst (aktuell: SSID “IOT”, Broker 10.200.0.207); Risiko durch “Client Isolation” im Schulnetz beachtet, Fallback: Mobile Hotspot für beide Geräte

Tabelle 7: Aufgetretene Probleme und deren Lösungen im Projektverlauf

7. Aktueller Status & offene Punkte

7.1 Was funktioniert

- Wiener-Linien-Live-Anzeige für Floridsdorf und Siebenhirten auf dem T-Display S3.
- Anwesenheitserkennung für Kilian und Marc per Reed-Kontakt, inkl. MQTT-Publishing und Home-Assistant Auto-Discovery.
- Home-Assistant-Dashboard “HomeKey Zentrale” mit Tile-Cards, Wetter-Badge und WG-Status-Spruch.
- MQTT-gesteuerter Ein/Aus-Schalter für das Display (Backlight GPIO 38).
- Docker-Infrastruktur mit Home Assistant, Mosquitto, ESPHome und Nginx.

7.2 Offene Punkte

- Patrik-Sensor (KY-021): Nachlöten von 5 V auf 3,3 V noch ausständig; bis dahin bleibt der Pin software-seitig deaktiviert bzw. die Invertierungslogik ist zu verifizieren.
- LED-Streifen (6 LEDs Anwesenheit auf GPIO 2, 35 LEDs Abfahrtszeiten auf GPIO 16): Verkabelung mit externem 5-V-Netzteil in Arbeit, FastLED-Firmware entworfen, aber noch nicht final in Main_code.ino integriert. Reparatur eines abgebrochenen Lötkontakts steht aus.
- Gemeinsame Masse (GND) zwischen Netzteil, beiden LED-Streifen und ESP32 muss beim Verlöten sichergestellt werden.
- Sobald die LEDs zuverlässig funktionieren, sollte entschieden werden, ob das Home-Assistent-Dashboard parallel weiterbestehen soll oder die LEDs es vollständig ersetzen.

7.3 Empfehlung für die nächsten Schritte

7. Patrik-Sensor auf 3,3 V umlöten und Invertierungslogik mit einem Magnet-Test verifizieren.
8. LED-Verkabelung gemäß Checkliste in Kapitel 2.4 fertigstellen.
9. FastLED-Firmware (6+35 LEDs) in Main_code.ino integrieren und testen.
10. Nach erfolgreichem Test: disabledPins[2] auf false setzen und Patrik-Sensor reaktivieren.

8. Anhang: Vollständiger Quellcode

8.1 Main_code.ino

```
#include <TFT_eSPI.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>
#include <PubSubClient.h>

// =====
// 1. NETZWERK KONFIGURATION
// =====
const char* ssid    = "IOT";
const char* password = "20tgmiot18";

const String urlFloridsdorf = "https://www.wienerlinien.at/ogd_realtime/monitor?stopId=4641";
const String urlSiebenhirten = "https://www.wienerlinien.at/ogd_realtime/monitor?stopId=4650";

// =====
// 2. MQTT KONFIGURATION (Broker & Themen)
// =====
const char* mqtt_server = "10.200.0.207";
const int  mqtt_port   = 1883;

const int numKeys = 3;
const char* keyNames[numKeys] = {"Kilian", "Marc", "Patrik"};
String stateTopics[numKeys]; // Wird im Setup dynamisch befüllt

// Display Switch MQTT Topics
const char* mqtt_switch_config_topic = "homeassistant/switch/homekey_display/config";
const char* mqtt_switch_state_topic  = "schluesselbrett/display/state";
const char* mqtt_switch_cmd_topic    = "schluesselbrett/display/set";
```

```

WiFiClient espMqttClient;
PubSubClient mqttClient(espMqttClient);

// =====
// 3. HARDWARE PIN DEFINITIONEN (SENSOREN)
// =====
// GPIO 13 = Kilian, GPIO 12 = Marc, GPIO 11 = Patrik
const int reedPins[numKeys] = {10, 12, 11};

// Patrik hat das KY-021 Modul -> JETZT AUF false DAMIT DIE LOGIK STIMMT!
const bool invertedPins[numKeys] = {false, false, false};

// Variablen fuer Entprellung (Debounce)
bool lastKeyStates[numKeys] = {false, false, false};
unsigned long lastDebounceTime[numKeys] = {0, 0, 0};
const unsigned long debounceDelay = 50;

// =====
// 4. DISPLAY CONFIGURATION
// =====
TFT_eSPI tft = TFT_eSPI();

// =====
// 5. CALLBACK: HOERT AUF DASHBOARD-BEFEHLE
// =====
void callback(char* topic, byte* payload, unsigned int length) {
    String message = "";
    for (unsigned int i = 0; i < length; i++) {
        message += (char)payload[i];
    }

    if (String(topic) == mqtt_switch_cmd_topic) {
        if (message == "ON") {
            digitalWrite(38, HIGH);
            mqttClient.publish(mqtt_switch_state_topic, "ON", true);
        }
        else if (message == "OFF") {
            digitalWrite(38, LOW);
            mqttClient.publish(mqtt_switch_state_topic, "OFF", true);
        }
    }
}

// =====
// HOME ASSISTANT AUTO-DISCOVERY LOGIK
// =====
void setupMQTTDiscovery() {
    for (int i = 0; i < numKeys; i++) {
        String configTopic = "homeassistant/binary_sensor/" + String(keyNames[i]) + "/config";
        stateTopics[i] = "schluesselbrett/" + String(keyNames[i]) + "/state";

        DynamicJsonDocument doc(1024);
        doc["name"] = String(keyNames[i]);
        doc["stat_t"] = stateTopics[i];
        doc["unique_id"] = "schluesselbrett_" + String(keyNames[i]);
        doc["dev_cla"] = "presence";
        doc["payload_on"] = "ON";
        doc["payload_off"] = "OFF";

        JsonObject device = doc.createNestedObject("device");
        device["identifiers"][0] = "schluesselbrett_sensoren_only";
        device["name"] = "Schluesselbrett Personen";
        device["model"] = "ESP32 Sensor-Modul";
        device["manufacturer"] = "Eigenbau";

        String payload;
        serializeJson(doc, payload);
        mqttClient.publish(configTopic.c_str(), payload.c_str(), true);
    }
}

```

```

}

DynamicJsonDocument switchDoc(1024);
switchDoc["name"] = "Display";
switchDoc["stat_t"] = mqtt_switch_state_topic;
switchDoc["cmd_t"] = mqtt_switch_cmd_topic;
switchDoc["icon"] = "mdi:monitor";
switchDoc["unique_id"] = "schluesselbrett_display_switch";
switchDoc["payload_on"] = "ON";
switchDoc["payload_off"] = "OFF";

JsonObject swDevice = switchDoc.createNestedObject("device");
swDevice["identifiers"][0] = "schluesselbrett_sensoren_only";

String swPayload;
serializeJson(switchDoc, swPayload);
mqttClient.publish(mqtt_switch_config_topic, swPayload.c_str(), true);
mqttClient.publish(mqtt_switch_state_topic, "ON", true);
}

void forcePublishAllStates() {
  for(int i = 0; i < numKeys; i++) {
    bool raw = (digitalRead(reedPins[i]) == LOW);
    bool currentState = invertedPins[i] ? !raw : raw;
    mqttClient.publish(stateTopics[i].c_str(), currentState ? "ON" : "OFF", true);
    lastKeyStates[i] = currentState;
  }
}

// =====
// SENSOR ABFRAGE
// =====
void checkSensors() {
  for (int i = 0; i < numKeys; i++) {
    bool raw = (digitalRead(reedPins[i]) == LOW);
    bool reading = invertedPins[i] ? !raw : raw;

    if (reading != lastKeyStates[i]) {
      if ((millis() - lastDebounceTime[i]) > debounceDelay) {
        lastKeyStates[i] = reading;
        if (mqttClient.connected()) {
          mqttClient.publish(stateTopics[i].c_str(), reading ? "ON" : "OFF", true);
        }
      }
      lastDebounceTime[i] = millis();
    }
  }
}

// =====
// MQTT RECONNECT
// =====
void reconnectMQTT() {
  if (mqttClient.connected()) return;

  String clientId = "ESP32Combined-";
  clientId += String(random(0xffff), HEX);

  if (mqttClient.connect(clientId.c_str())) {
    setupMQTTDiscovery();
    forcePublishAllStates();
    mqttClient.subscribe(mqtt_switch_cmd_topic);
  }
}

// =====
// DISPLAY HILFSFUNKTIONEN
// =====

```

```

String fixUmlaute(String s) {
    s.replace("ae", "ae"); s.replace("Ae", "Ae");
    s.replace("oe", "oe"); s.replace("Oe", "Oe");
    s.replace("ue", "ue"); s.replace("Ue", "Ue");
    s.replace("ss", "ss");
    return s;
}

uint16_t getCountdownColor(int cd) {
    if (cd < 0) return TFT_DARKGREY;
    if (cd <= 2) return TFT_RED;
    if (cd <= 4) return TFT_YELLOW;
    return TFT_GREEN;
}

void drawRow(int yPos, String direction, int cd1, int cd2) {
    tft.setTextDatum(TL_DATUM);
    tft.setTextColor(TFT_GOLD);
    tft.drawString(direction, 8, yPos, 4);

    tft.setTextColor(TFT_ORANGE);
    tft.drawString("U6", 8, yPos + 26, 2);

    tft.setTextDatum(TR_DATUM);
    tft.setTextColor(getCountdownColor(cd1));
    String z1 = (cd1 >= 0) ? String(cd1) : "--";
    tft.drawString(z1, 280, yPos, 7);

    tft.setTextColor(getCountdownColor(cd2));
    String z2 = (cd2 >= 0) ? String(cd2) : "--";
    tft.drawString(z2, 318, yPos + 8, 4);
}

bool fetchDepartures(const String& url, String& direction, int& cd1, int& cd2) {
    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    http.setTimeout(10000);
    http.begin(client, url);

    int httpCode = http.GET();
    if (httpCode != 200) { http.end(); return false; }

    DynamicJsonDocument doc(30000);
    DeserializationError error = deserializeJson(
        doc, http.getStream(), DeserializationOption::NestingLimit(15)
    );
    http.end();

    if (error) {
        return false;
    }

    JsonObject lineObj = doc["data"]["monitors"][0]["lines"][0];
    direction = fixUmlaute(String(lineObj["towards"] | "Unbekannt"));
    JSONArray deps = lineObj["departures"]["departure"];
    cd1 = (deps.size() > 0) ? (int)deps[0]["departureTime"]["countdown"] : -1;
    cd2 = (deps.size() > 1) ? (int)deps[1]["departureTime"]["countdown"] : -1;
    return true;
}

// =====
// SETUP & LOOP
// =====
void setup() {
    Serial.begin(115200);

```

```

pinMode(38, OUTPUT);
digitalWrite(38, HIGH);

for (int i = 0; i < numKeys; i++) {
  stateTopics[i] = "schluesselbrett/" + String(keyNames[i]) + "/state";
  pinMode(reedPins[i], invertedPins[i] ? INPUT : INPUT_PULLUP);
}

tft.init();
tft.setRotation(3);
tft.fillScreen(TFT_BLACK);
tft.setTextDatum(MC_DATUM);
tft.setTextColor(TFT_WHITE);
tft.drawString("Verbinde WiFi...", tft.width() / 2, tft.height() / 2, 2);

WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
}

mqttClient.setBufferSize(1024);
mqttClient.setServer(mqtt_server, mqtt_port);
mqttClient.setCallback(callback);
reconnectMQTT();

tft.fillScreen(TFT_BLACK);
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    WiFi.begin(ssid, password);
    delay(3000);
    return;
  }

  if (!mqttClient.connected()) reconnectMQTT();
  mqttClient.loop();

  checkSensors();

  String dir1, dir2;
  int cd1_1 = -1, cd1_2 = -1;
  int cd2_1 = -1, cd2_2 = -1;

  bool ok1 = fetchDepartures(urlFloridsdorf, dir1, cd1_1, cd1_2);
  bool ok2 = fetchDepartures(urlSiebenhirten, dir2, cd2_1, cd2_2);

  tft.fillScreen(TFT_BLACK);
  tft.setTextDatum(TC_DATUM);
  tft.setTextColor(TFT_WHITE);
  tft.drawString("Jaegerstrasse", tft.width() / 2, 2, 2);
  tft.drawFastHLine(6, 20, tft.width() - 12, 0x3186);

  if (ok1) {
    drawRow(28, dir1, cd1_1, cd1_2);
  } else {
    tft.setTextDatum(TL_DATUM);
    tft.setTextColor(TFT_DARKGREY);
    tft.drawString("Floridsdorf - keine Daten", 8, 28, 2);
  }

  tft.drawFastHLine(6, 85, tft.width() - 12, 0x1863);

  if (ok2) {
    drawRow(90, dir2, cd2_1, cd2_2);
  } else {

```

```
tft.setTextDatum(TL_DATUM);  
tft.setTextColor(TFT_DARKGREY);  
tft.drawString("Siebenhirten - keine Daten", 8, 90, 2);  
}  
  
unsigned long waitUntil = millis() + 30000;  
while (millis() < waitUntil) {  
  if (!mqttClient.connected()) reconnectMQTT();  
  mqttClient.loop();  
  checkSensors();  
  delay(50);  
}  
}
```